

SIMATS ENGINEERING
(SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2
CI/CD Pipeline Design Exercise

*Problem Statement 33: AI-Powered Smart Municipal Asset Management
and Maintenance Platform*

Course Code	CSA10
Course Title	Software Engineering
CO Assessed	CO4
Assessment	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage	20%
Total Marks	25
Student Name	Jaswanth S
Register Number	192525214
Duration	60 minutes

Table of Contents

Section	Page
1. Assigned Problem Statement	3
1.1 Industry Context	3
1.2 Problem Statement	3
1.3 Objectives	3
1.4 Expected Outcomes	3
2. CI/CD Requirement Analysis	4
2.1 Key CI/CD Drivers	4
2.2 Non-Functional Requirements Influencing the Pipeline	4
3. CI/CD Pipeline Architecture and Workflow	5
3.1 Pipeline Diagram (Stage Flow)	5
3.2 Stage-wise Workflow Explanation	5
4. Tools and Technology Selection	7
5. Automated Testing Strategy	8
5.1 Additional Quality Practices	8
6. Deployment Strategy	9
6.1 Blue-Green Deployment for Production	9
6.2 Canary Option for High-Risk Changes	9
7. Security, Quality Checks, Notifications and Rollback Mechanisms	9
7.1 Security Practices	9
7.2 Quality Gates	9
7.3 Notification Mechanisms	10
7.4 Rollback Mechanism	10
8. Conclusion	11
Annexure – Evaluation Rubric	12

1. Assigned Problem Statement

1.1 Industry Context

Municipal corporations manage a wide range of public assets such as roads, parks, streetlights, drainage systems, public buildings, and water infrastructure. Efficient management of these assets is essential to improving the quality of public services and reducing long-term maintenance expenditure. The convergence of Artificial Intelligence (AI), Geographic Information Systems (GIS), Internet of Things (IoT) sensors, and cloud computing now enables predictive maintenance and centralized, data-driven infrastructure management for city administrations.

1.2 Problem Statement

Municipal assets often deteriorate due to delayed maintenance and fragmented management systems. Manual inspections make it difficult for civic bodies to prioritize repairs effectively, leading to inefficient budget allocation and citizen dissatisfaction. The proposed AI-Powered Smart Municipal Asset Management and Maintenance Platform must maintain a digital inventory of public assets, continuously monitor asset conditions using IoT sensors, predict maintenance requirements using machine learning, prioritize repair schedules, and generate infrastructure performance reports — thereby improving service quality while reducing maintenance costs.

1.3 Objectives

1. Digitize municipal asset records into a centralized, searchable inventory.
2. Monitor asset conditions continuously through IoT sensor networks.
3. Predict maintenance requirements using AI/ML-based failure prediction models.
4. Prioritize repair schedules based on risk, cost, and citizen impact.
5. Integrate GIS-based asset mapping for spatial visualization.
6. Generate maintenance analytics and infrastructure performance dashboards.
7. Improve utilization of existing infrastructure resources.
8. Enhance public service delivery and citizen satisfaction.

1.4 Expected Outcomes

- Improved infrastructure reliability and reduced asset downtime.
- Reduced overall maintenance and operational costs.
- Better utilization of public assets across the city.
- Faster, data-driven maintenance planning and scheduling.
- Enhanced municipal service quality for citizens.
- Centralized, data-driven infrastructure management.
- Increased operational efficiency for municipal staff.
- Higher citizen satisfaction through timely repairs and transparency.

2. CI/CD Requirement Analysis

The platform is a multi-component system comprising a citizen/staff-facing web dashboard, a GIS mapping service, an IoT data-ingestion microservice, an AI/ML prediction service, and a relational/spatial database layer. Because these components evolve independently and must integrate with live IoT sensor feeds, a robust CI/CD pipeline is required to ensure rapid, reliable, and safe delivery of updates without disrupting critical municipal operations.

2.1 Key CI/CD Drivers

- Frequent, incremental updates to the AI prediction model as new sensor data is collected.
- Multiple independent microservices (asset inventory, GIS, IoT ingestion, analytics) requiring isolated build and test cycles.
- High reliability requirement — downtime affects real-time monitoring of public infrastructure (e.g., drainage overflow alerts).
- Strict data security requirements due to handling of citizen and government infrastructure data.
- Need for zero/near-zero downtime deployment since the system supports continuous IoT data streams.
- Compliance and auditability requirements typical of government/municipal IT systems.

2.2 Non-Functional Requirements Influencing the Pipeline

1. Scalability — pipeline and infrastructure must scale to handle growing numbers of IoT sensors across the city.
2. Security — automated vulnerability and dependency scanning at every build.
3. Reliability — automated rollback in case of failed deployment or performance regression.
4. Traceability — every deployment must be traceable to a commit, build, and test report for audit purposes.
5. Maintainability — pipeline-as-code so that DevOps configuration is version-controlled alongside application code.

3. CI/CD Pipeline Architecture and Workflow

The pipeline below illustrates the complete journey of a code change from developer commit through to production deployment and monitoring, covering source control, build, automated testing, code quality/security analysis, packaging, staged deployment, and rollback.

3.1 Pipeline Diagram (Stage Flow)

Code Commit (Developer)	Source Code Management (GitHub)	Build (Maven/ Docker)	Automated Testing (JUnit/ Selenium)	Code Quality & Security Scan (SonarQube/ Snyk)	Artifact Packaging (Docker/ Nexus)	Deploy to Staging (Kubernetes)	Staging Validation (UAT)	Deploy to Production (Blue-Green)	Monitoring & Rollback (Prometheus)
-----------------------------------	---	------------------------------------	--	---	---	--	------------------------------------	---	--

Figure 1: End-to-end CI/CD pipeline for the Smart Municipal Asset Management Platform

3.2 Stage-wise Workflow Explanation

Stage 1 — Code Commit

A developer commits code (e.g., a new maintenance-prediction algorithm or dashboard feature) to a feature branch and raises a pull request against the 'develop' branch.

Stage 2 — Source Code Management (SCM)

GitHub hosts the repository with branch protection rules. A webhook automatically triggers the CI pipeline on every push or pull request.

Stage 3 — Build

The CI server checks out the code, resolves dependencies (Maven for backend Java services, npm for the React-based dashboard), and compiles the application into deployable artifacts.

Stage 4 — Automated Testing

Unit and integration tests execute automatically to validate business logic such as asset-condition scoring, maintenance-priority calculation, and IoT data-ingestion correctness.

Stage 5 — Code Quality and Security Analysis

SonarQube analyzes code smells, duplication, and complexity while Snyk/OWASP Dependency-Check scans for vulnerable dependencies and container CVEs. A quality gate blocks the pipeline if thresholds are not met.

Stage 6 — Artifact Packaging

On passing all quality gates, the application is containerized using Docker and the image is pushed to a private container registry (Nexus/Docker Hub) with a version tag.

Stage 7 — Deployment to Staging

The container image is automatically deployed to a Kubernetes-based staging environment that mirrors production, using Helm charts for consistent configuration.

Stage 8 — Staging Validation / UAT

Automated end-to-end tests (Selenium/Cypress) and manual User Acceptance Testing by municipal officials validate the GIS dashboard, repair-prioritization workflow, and report generation.

Stage 9 — Deployment to Production

On manual approval, a Blue-Green (or Canary) deployment strategy releases the new version to production with minimal risk and near-zero downtime.

Stage 10 — Monitoring, Notification and Rollback

Prometheus and Grafana continuously monitor application health, IoT data throughput, and infrastructure KPIs. Automated alerts notify the DevOps team via Slack/Teams/Email, and an automatic rollback is triggered if health checks fail post-deployment.

4. Tools and Technology Selection

The following table lists the recommended tool for each pipeline stage along with the justification for its selection, based on the platform's requirements for scalability, IoT integration, GIS support, and government-grade security.

Stage	Recommended Tool(s)	Justification
Source Code Management	GitHub / GitLab	Distributed version control, pull-request review workflow, native webhook triggers for CI
CI Orchestration	Jenkins / GitHub Actions	Mature plugin ecosystem, YAML-based pipeline-as-code, supports parallel and multi-stage jobs
Build & Dependency Mgmt	Maven (Java) / npm (Node.js)	Standardized build lifecycle, dependency resolution, reproducible artifacts
Containerization	Docker	Consistent runtime across dev, IoT edge, staging and production environments
Unit & Integration Testing	JUnit, Mockito, Postman/Newman	Automated verification of business logic (asset scoring, maintenance prediction APIs)
UI / E2E Testing	Selenium / Cypress	Validates GIS dashboard and citizen-facing web portal workflows
Code Quality Analysis	SonarQube	Detects code smells, duplication, and enforces quality gates before merge
Security / Dependency Scan	Snyk / OWASP Dependency-Check	Identifies vulnerable libraries and container image CVEs early
Artifact Repository	Nexus / Docker Hub (private)	Central, versioned storage for build artifacts and container images
Container Orchestration	Kubernetes (K8s)	Auto-scaling for IoT sensor data ingestion spikes; supports blue-green/canary deployment
Infrastructure as Code	Terraform / Helm	Reproducible provisioning of cloud, GIS, and IoT-gateway infrastructure
Monitoring & Logging	Prometheus + Grafana, ELK Stack	Real-time asset/sensor health and pipeline observability with alerting
Notification	Slack / MS Teams / Email (SMTP)	Immediate build/deployment/failure alerts to DevOps and municipal IT teams
Secrets Management	HashiCorp Vault / K8s Secrets	Secure storage of GIS API keys, DB credentials, IoT gateway tokens

5. Automated Testing Strategy

A layered testing strategy is integrated into the pipeline so that defects are caught as early as possible ('shift-left' testing), minimizing the risk of faulty releases reaching municipal staff or citizens.

Testing Level	Scope in the Platform	Tools	Pipeline Trigger Point
Unit Testing	Asset scoring algorithm, maintenance prediction model functions, GIS coordinate utilities	JUnit, PyTest (for ML prediction service)	On every commit / pull request
Integration Testing	IoT sensor data ingestion API ↔ database, GIS mapping service ↔ asset inventory service	Postman/Newman, TestContainers	After successful build stage
Code Quality Gate	Complexity, duplication, coding standard compliance	SonarQube Quality Gate	Post-build, pre-packaging
Security Testing	Dependency CVEs, container image vulnerabilities, static code security issues	OWASP Dependency-Check, Snyk, Trivy	Post-build, pre-packaging
System / E2E Testing	End-to-end citizen service request flow, dashboard rendering, alert generation	Selenium, Cypress	In staging environment
Performance/Load Testing	IoT sensor data burst handling, concurrent municipal-staff dashboard access	JMeter, k6	In staging environment before promotion
User Acceptance Testing (UAT)	Municipal officials validate repair-prioritization workflow and reports	Manual + automated regression suite	Staging sign-off gate

5.1 Additional Quality Practices

- Test coverage threshold of at least 80% enforced via the SonarQube quality gate before merge is allowed.
- Contract testing between the IoT-ingestion microservice and the analytics/prediction service to prevent breaking changes.
- Regression test suite automatically re-run before every production deployment.
- Mutation testing periodically applied to critical modules (e.g., repair-prioritization algorithm) to assess test-suite effectiveness.

6. Deployment Strategy

The platform uses a three-tier environment strategy — Development, Testing/Staging, and Production — to progressively validate changes before they reach citizens and municipal staff.

Environment	Purpose	Deployment Strategy	Data Used
Development	Feature development and unit testing by individual developers	Continuous auto-deploy on every commit to feature branch	Synthetic/sample asset & sensor data
Testing / Staging	Integration, system, performance and UAT before release	Automated deploy on merge to 'develop'/'release' branch after quality gate pass	Anonymized copy of production-like data
Production	Live municipal operations serving citizens and staff	Blue-Green / Canary deployment with manual approval gate	Live IoT sensor feeds and production asset database

6.1 Blue-Green Deployment for Production

Production releases use a Blue-Green deployment model: the new version ('Green') is deployed alongside the current live version ('Blue') within Kubernetes. Traffic is switched to Green only after automated smoke tests and health checks pass, allowing an instantaneous rollback to Blue if issues are detected.

6.2 Canary Option for High-Risk Changes

For high-risk changes such as updates to the AI maintenance-prediction model, a canary release routes a small percentage of traffic (e.g., 5–10%) to the new version first, with automated monitoring of prediction accuracy and system performance before full rollout.

7. Security, Quality Checks, Notifications and Rollback Mechanisms

7.1 Security Practices

- Static Application Security Testing (SAST) via SonarQube integrated at every build.
- Software Composition Analysis (SCA) using Snyk/OWASP Dependency-Check to flag vulnerable third-party libraries.
- Container image scanning with Trivy prior to pushing to the registry.
- Secrets (database credentials, GIS API keys, IoT gateway tokens) stored in HashiCorp Vault / Kubernetes Secrets — never hard-coded.
- Role-Based Access Control (RBAC) on the CI/CD server and Kubernetes cluster restricting who can approve production deployments.
- TLS encryption for all IoT sensor-to-cloud and service-to-service communication.

7.2 Quality Gates

- Build fails automatically if unit test coverage falls below the defined threshold.
- SonarQube quality gate blocks merge on critical code smells or high-severity vulnerabilities.
- Manual approval gate required before promotion from Staging to Production.

7.3 Notification Mechanisms

- Slack/MS Teams channel integration for real-time build, test, and deployment status.
- Email alerts to the DevOps and municipal IT teams on pipeline failure or successful production release.
- Dashboard (Grafana) alerts for post-deployment anomalies in IoT data throughput or API latency.

7.4 Rollback Mechanism

- Automatic rollback to the previous stable container image if post-deployment health checks fail within a defined monitoring window (e.g., 10 minutes).
- Kubernetes 'kubectl rollout undo' integrated into the pipeline as a one-click/automated rollback action.
- Database migrations designed to be backward-compatible so that rollback does not break data integrity.
- Immutable, versioned artifacts in Nexus/Docker Hub ensure any previous release can be redeployed on demand.

8. Conclusion

The proposed CI/CD pipeline for the AI-Powered Smart Municipal Asset Management and Maintenance Platform automates the complete journey from code commit to production deployment, integrating rigorous automated testing, code quality analysis, and security scanning at every stage. By adopting a three-tier environment strategy with Blue-Green/Canary deployment and automated rollback, the pipeline ensures high reliability and near-zero downtime — critical for a system that continuously monitors public infrastructure. Overall, this DevOps approach enables the municipal corporation to deliver faster, more reliable software updates while maintaining the security, quality, and auditability standards expected of government infrastructure systems.

Annexure – Evaluation Rubric

The following rubric, as provided in the assessment tool, will be used for evaluation of this submission.

Criteria	Marks
Requirement & Pipeline Analysis (15%)	/5
Pipeline Architecture & Workflow (25%)	/4
Tools & Technology Selection (20%)	/4
Automated Testing & Quality Integration (15%)	/4
Deployment, Security & Rollback Strategy (15%)	/4
Pipeline Diagram & Documentation (10%)	/4
TOTAL	/25